

Introducción a la Modulación Digital

Universidad ORT Uruguay

Prof. Fernando Paganini

Versión electrónica elaborada por
Andrés Ferragut y Fabián Kozynski

Revisado en: Agosto 2025

Índice

1. Introducción	2
1.1. Motivación de la transmisión digital	3
1.2. Diagrama general de un sistema de transmisión digital	5
1.3. Conceptos básicos de codificación	5
1.3.1. Codificación de fuente	5
1.3.2. Codificación de canal	8
2. Procesos aleatorios	10
A. Apéndice	11

Capítulo 1

Introducción

El objetivo de este curso es analizar las técnicas básicas para la *transmisión digital de datos* sobre un canal imperfecto. Para ello, haremos un fuerte uso de las técnicas de análisis de señales en tiempo discreto y continuo (teoremas de muestreo, Transformada de Fourier, etc.), y las extenderemos para realizar modelos de transmisión adecuados a las señales digitales.

En primer lugar, corresponde definir nuestro objeto de trabajo:

Definición (Señal digital de tiempo discreto). *Una señal digital es una secuencia $\{a_k : k \in \mathbb{Z}\}$ de símbolos, cada uno de ellos perteneciente a un alfabeto finito:*

$$a_k \in \{s_1, \dots, s_M\}$$

siendo M el tamaño del alfabeto.

Cuando $M = 2$, el alfabeto se denomina binario y cada símbolo puede interpretarse como un 0 o un 1, permitiendo así la transmisión básica de la información digital. Sin embargo, como veremos más adelante, puede ser relevante considerar alfabetos de mayor cantidad de símbolos, permitiendo enviar mayor cantidad de “bits por símbolo”, es decir cada símbolo contiene una mayor cantidad de “información”.

Notemos además que los símbolos no son necesariamente 0 y 1. Por ejemplo, para un alfabeto binario veremos que puede ser interesante codificar los símbolos como 1 y -1 en algunas aplicaciones, de manera de obtener señales de valor medio nulo.

El objetivo entonces de un sistema de comunicación digital es transmitir, sobre un canal imperfecto, una señal digital de tiempo discreto y *reconstruir*, a la salida, una señal digital $\{\hat{a}_k\}$ de manera tal de que *maximizar la cantidad de símbolos reconstruidos de manera correcta*. Dicho de otro modo, si el sistema de transmisión, modulación, canal y detección introducen perturbaciones aleatorias, que la probabilidad de *error de símbolo*, $P(a_k \neq \hat{a}_k)$ sea (en régimen) lo menor posible.

Como se intuye de la ecuación anterior, aparecen dos conceptos clave a tener en cuenta en el presente curso:

- Es útil trabajar con *señales aleatorias*. Esto es porque no conocemos a priori *qué es lo que se va a transmitir*, y quizás solo propiedades estadísticas de la señal. A su vez, todo canal de comunicación está sujeto a *ruido* que es intrínsecamente aleatorio. Esto requiere familiaridad con el concepto de *proceso estocástico* que definiremos en el Capítulo 2, así como con las reglas básicas de probabilidad, que se recuerdan en el Apéndice ??.

- Por otra parte, al tratarse de análisis de señales, tanto en tiempo discreto como tiempo continuo, se requiere familiaridad con los conceptos de sistemas lineales invariantes en el tiempo (que servirán para modelar el canal, entre otras cosas), así como la *Transformada de Fourier*, tanto en su versión discreta como continua. En particular veremos cómo las propiedades estadísticas de una señal se relacionan, mediante la transformada de Fourier, con su *densidad espectral de potencia*, concepto que definiremos también en el Capítulo 2.

1.1. Motivación de la transmisión digital

La primera pregunta que debemos contestar es por qué puede ser conveniente transmitir datos de manera digital, esto es, siguiendo un alfabeto finito, en lugar de transmitir o intentar reproducir una forma de onda, como se vio en la modulación analógica. Si bien esta última fue el primer mecanismo masivo de transmisión de datos a larga distancia, y aún se utiliza hoy en múltiples aplicaciones (radio comercial, control de tráfico aéreo, etc.), debemos recordar que el *primer sistema de transmisión eléctrica de datos* corresponde a la telegrafía: esto es, el envío de mensajes sobre un cable de cobre (cable telegráfico). Dichos mensajes correspondían típicamente a mensajes de texto (letras y números), que eran *codificados* como pulsos de distinta duración (cortos y largos). Esto dio lugar al primer ejemplo de código para la transmisión eléctrica: el código Morse (1844, c.f. [telégrafía eléctrica](#)).

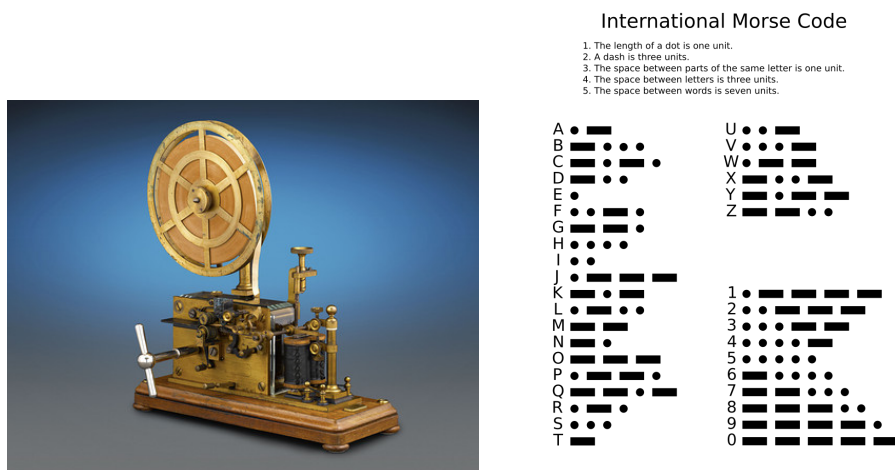


Figura 1.1. Telégrafo de Morse.

Para entender las ventajas de la comunicación digital, es bueno remitirse primero a un esquema básico de comunicación analógico, como el mostrado en la Figura 1.2. Allí, la fuente de información (ej. audio) es transformada en una señal eléctrica continua $x(t)$ por un transductor (ej. micrófono). El sistema de modulación intentará transmitir dicha señal por un medio físico, recibiendo $\hat{x}(t)$, señal que luego es enviada a otro transductor (ej. parlante) para enviarla al receptor. El objetivo de este proceso es que la señal $\hat{x}(t)$ reproduzca lo más fielmente posible la señal original $x(t)$, en todo momento del tiempo. Sin embargo, esto resulta imposible debido a que el canal siempre introduce atenuación, limitaciones de ancho de banda, potencia de transmisión y ruido que alteran la recepción (en algunos casos hasta volverla inservible).

En la transmisión digital, se transmiten símbolos a_k construyendo una señal real $x(t)$ que depende de dichos símbolos. Si la señal recibida $\hat{x}(t)$ es suficientemente inteligible, podemos decodificar el símbolo $\hat{a}_k$ con *muy alta probabilidad* de que sea exactamente igual a a_k . En ese caso, la reconstrucción de la señal es perfecta (con alta probabilidad). Es decir, *no hay pérdida de información por atravesar el canal*.

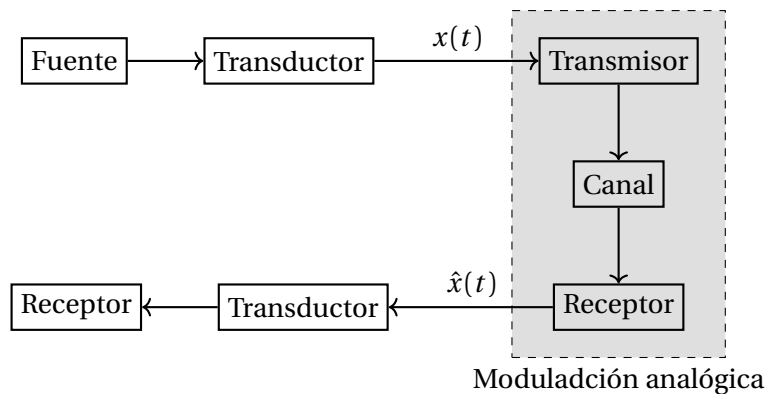


Figura 1.2. Diagrama general de un sistema de transmisión analógico.

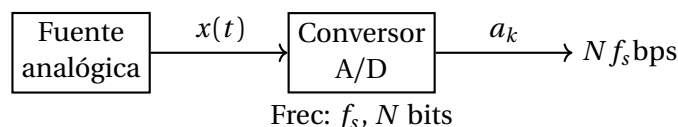


Figura 1.3. Conversión analógica a digital.

Esto vuelve la transmisión mucho más robusta al ruido e interferencia. El objetivo principal de este curso es entender precisamente cómo realizar este proceso.

Complementario a lo anterior existen dos fuertes razones para analizar la transmisión digital de datos:

1. En primer lugar, en los últimos años, casi toda la información se almacena en forma digital (texto, fotos, video). En ese sentido, la información ya se encuentra en un alfabeto de símbolos (ej. binario) y no tiene sentido plantearse pasar por una etapa analógica, salvo para efectivamente transmitir sobre el medio físico.
2. En otros casos, la señal analógica se convierte en una señal digital mediante un convertor analógico/digital (A/D) (Figura 1.3) Aquí se cumplen dos etapas:
 - a) Se *muestrea la señal* a una frecuencia de muestreo f_s (muestras por segundo).
 - b) Se *cuantifica la señal* pasando el valor analógico a una cierta cantidad de símbolos (bits). Por ejemplo, las señales de audio se muestrean a $f_s = 44100\text{Hz}$ y se digitalizan utilizando 16 bits de resolución para cada canal estéreo. El resultado es una secuencia de muestras a_k de $\approx 1,41\text{Mbps}$.

En el segundo caso, por el Teorema de Nyquist del muestreo, sabemos que si la señal $x(t)$ tiene ancho de banda B , entonces obteniendo muestras a frecuencia $f_s > 2B$ es posible reconstruir la señal de manera *exacta* a partir de las muestras $x(kT_s)$ con $T_s = 1/f_s$. Sin embargo, como las muestras están cuantificadas, $a_k \approx x(kT_s)$ pero no es exacto, introduciendo el llamado *ruido de cuantificación*. Este ruido es menor a mayor cantidad de bits utilizados para representar las muestras. Sin embargo, si la transmisión digital permite reconstruir las muestras a_k exactamente, es probable que dicho ruido de cuantificación sea menor que el que introduce un esquema de modulación analógico, por lo cual la señal se recibe de mejor manera. A su vez, con los esquemas de modulación modernos, es posible aprovechar mejor el ancho de banda mandando más información (bits) sobre el mismo, por lo cual podemos realizar transmisiones de calidad.

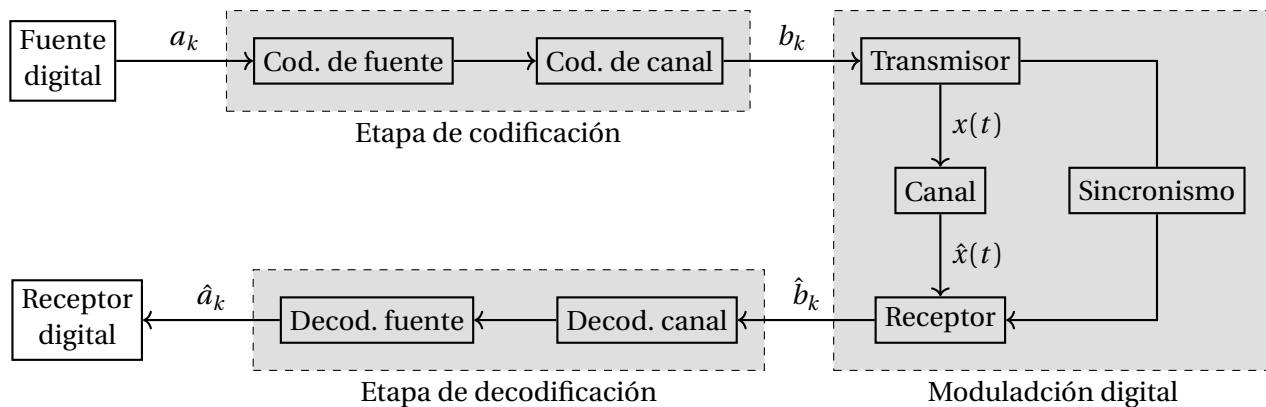


Figura 1.4. Diagrama general de un sistema de transmisión digital.

1.2. Diagrama general de un sistema de transmisión digital

El diagrama general de un sistema de transmisión digital se muestra en la Figura 1.4. Allí partimos de una fuente ya digital que genera símbolos $\{a_k\}$, y se pasa por dos etapas de codificación que explicaremos. Con ello se genera una nueva señal digital $\{b_k\}$ que se enviará por el medio físico. Para ello, se requiere una etapa de *modulación digital*, en la cual los símbolos b_k se convierten en una señal analógica adecuada, se envían por el canal, y luego son reconstruidos por un *detector* digital, que obtiene una secuencia $\hat{b}_k$. El objetivo de esta etapa es lograr que $P(\hat{b}_k = b_k)$ sea máxima, de este modo permitiendo reconstruir la señal. Luego esta se decodifica por el proceso inverso a la codificación para obtener $\hat{a}_k$ y enviar esta señal al receptor.

El objetivo de este curso es analizar la etapa de *modulación digital*, es decir dada una secuencia de símbolos $\{b_k\}$, cómo construir una señal analógica $x(t)$ que contenga la información de dichos símbolos y sea adecuada para su transmisión por el medio físico (radio, par de cobre, fibra óptica). Luego de su transmisión, la señal alterada deberá ser detectada y de allí reconstruiremos una secuencia de símbolos con baja probabilidad de error. El término *modulación digital* hace referencia a que la señal discreta *modula* características de la señal analógica. Notemos además que este proceso requiere un bloque adicional: el *sincronismo*, para que el receptor sepa a qué velocidad se transmite la información y cuál es el mejor momento del tiempo para muestrear la señal $\hat{x}(t)$ de manera de recuperar $\hat{b}_k$.

Si bien en este curso no trabajaremos sobre la etapa de *codificación*, es importante entender el rol que cumplen las mismas previo a la transmisión. Introducimos ahora los principales conceptos.

1.3. Conceptos básicos de codificación

Como se observa en la Figura 1.4, existen dos etapas: la *codificación de fuente* y la *codificación de canal*, con sus respectivos decodificadores.

1.3.1. Codificación de fuente

El objetivo de la codificación de fuente es *eliminar redundancia* de los datos, produciendo una secuencia de símbolos con menor cantidad de “bits por símbolo”, de manera de requerir menor velocidad de transmisión de información. Existen dos tipos:

Codificación sin pérdida: Aquí la secuencia de símbolos generada contiene toda la información de la señal $\{a_k\}$, pero *comprimida*. Esto es, utilizando propiedades estadísticas de la señal, se logra representar la misma información con menos bits. Un ejemplo de esto es el algoritmo [Lempel-Ziv](#), utilizado en la compresión “.zip”.

Codificación con pérdida: Aquí la secuencia de símbolos generada contiene *menos información* que la original, eliminando por ejemplo partes que no son requeridas por el receptor. Ejemplos de esto son la compresión de imágenes JPEG, la compresión de video MPEG y la codificación de audio MP3. En estos casos, la señal reconstruida es una “buena aproximación” de la información original. Qué quiere decir “buena aproximación” depende de la aplicación, pero a modo de ejemplo el MP3 permite reducir la velocidad de audio mencionada anteriormente (1,41Mbps) a 192kbps o menos sin que el usuario detecte mayormente la diferencia.

Ejemplo 1.1 (Codificación sin pérdida). *Consideremos una fuente de datos que genera símbolos independientes con la siguiente estadística:*

$$a_k \in \{0, 1, 2, 3\} \text{ con distribución } \begin{cases} P(a_k = 0) = 1/2 \\ P(a_k = 1) = 1/4 \\ P(a_k = 2) = 1/8 \\ P(a_k = 3) = 1/8 \end{cases}$$

En este caso, una codificación sencilla sería tomar el código:

$$\begin{cases} a_k = 0 \mapsto \tilde{a}_k = 00 \\ a_k = 1 \mapsto \tilde{a}_k = 01 \\ a_k = 2 \mapsto \tilde{a}_k = 10 \\ a_k = 3 \mapsto \tilde{a}_k = 11 \end{cases}$$

Observemos que el código anterior requiere en promedio 2 bits por símbolo (ya que todos los códigos son de 2 bits). ¿Podemos hacerlo mejor? Consideremos el siguiente código, debido a [Huffman](#):

$$\begin{cases} a_k = 0 \mapsto \tilde{a}_k = 0 \\ a_k = 1 \mapsto \tilde{a}_k = 10 \\ a_k = 2 \mapsto \tilde{a}_k = 110 \\ a_k = 3 \mapsto \tilde{a}_k = 111 \end{cases}$$

Notemos que el código anterior es un código de prefijo (cada símbolo empieza de manera diferente), lo que permite que sea invertible. Es decir si por ejemplo recibimos:

$$0101001110110111 \mapsto 0\ 10\ 10\ 0\ 111\ 0\ 110\ 111 \mapsto 01103023$$

Calculemos el largo medio de la palabra enviada de la siguiente forma, donde $\ell(\cdot)$ representa el largo del código:

$$\begin{aligned} \bar{L} &= P(a_k = 0)\ell(0) + P(a_k = 1)\ell(10) + P(a_k = 2)\ell(110) + P(a_k = 3)\ell(111) \\ &= \left(\frac{1}{2}\right)(1) + \left(\frac{1}{4}\right)(2) + \left(\frac{1}{8}\right)(3) + \left(\frac{1}{8}\right)(3) = 1,75 < 2 \end{aligned}$$

Esto quiere decir que en el largo plazo, una secuencia de N símbolos podrá ser representada por $\approx 1,75N$ bits (en lugar de $2N$), permitiendo un ahorro.

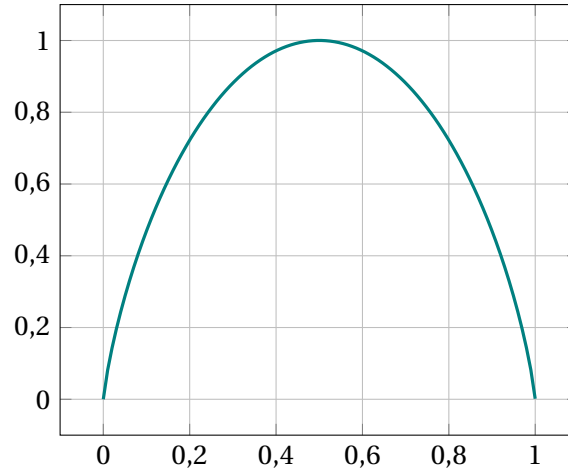


Figura 1.5. Entropía de una fuente binaria. Notar que el máximo se alcanza para $p = 1/2$, es decir, distribución uniforme.

La clave del ejemplo anterior está en elegir palabras más largas para símbolos menos frecuentes. Notemos que esto mismo ya había sido advertido por Morse en su código, como se ve en la figura 1.1. Surgen entonces las siguientes preguntas: ¿cuál es el mínimo largo medio posible de codificación? ¿Es alcanzable?

En 1948, [Claude Shannon](#) establece la teoría matemática de transmisión de información, introduciendo el concepto de *entropía* de una fuente. Dicha entropía representa la cantidad de información contenida en una fuente (considerando sus propiedades estadísticas). Para una fuente X que genera símbolos independientes $\{a_k\}$, la misma se define como:

$$H(X) = - \sum_k p_k \log_2(p_k)$$

siendo $\log_2$ en logaritmo en base 2, lo que hace que la entropía quede en bits/símbolo.

Calculemos la entropía de la fuente anterior:

$$H(X) = \left(\frac{1}{2}\right)(1) + \left(\frac{1}{4}\right)(2) + \left(\frac{1}{8}\right)(3) + \left(\frac{1}{8}\right)(3) = 1,75.$$

Es decir, recuperamos exactamente el largo medio del código de Huffman anterior.

Ejemplo 1.2 (Fuente binaria). Supongamos que los símbolos son binarios $a_k \in \{0, 1\}$ con $P(a_k = 1) = p = 1 - P(a_k = 0)$ e independientes. En ese caso la entropía queda (como función de p):

$$H(p) = -p \log_2(p) - (1-p) \log_2(1-p), \quad (1.1)$$

cuya gráfica puede verse en la Figura 1.5

El Teorema de Shannon muestra que para una fuente dada, su entropía es la mínima cantidad de bits por símbolo alcanzable:

Teorema 1.1 (Shannon, 1948). Sea una fuente de símbolos $\{a_k\}$ y su entropía H_a . Entonces es posible agrupar los símbolos de a bloques de tamaño N y codificarlos en palabras de L_N bits de manera que el largo medio de palabra cumpla:

$$NH_a \leq \bar{L}_N \leq NH_a + 1$$

Como cada bloque codifica N símbolos, la cantidad de bits por símbolo será entonces:

$$H_a \leq \frac{\bar{L}}{N} \leq H_a + 1/N$$

Además, no existe ningún código que logre mejorar esta cota.

La entropía de una fuente nos entonces da la mínima cantidad de bits por símbolo que podemos utilizar, es decir, el límite de la compresión alcanzable. Si volvemos al Ejemplo 1.1, vemos que precisamente el código de Huffman alcanza la mínima entropía posible, incluso con $N = 1$. Para otras distribuciones puede ser necesario utilizar un largo mayor de bloque para aproximar la entropía.

En el Ejemplo 1.2, vemos que la entropía es máxima e igual a 1 cuando los símbolos son *equiprobables* ($p = 1/2$). En ese caso, debemos utilizar 1 bit por símbolo y la señal es incompresible. Precisamente, el código de Huffman descrito logra que la fuente original genere cantidades equiprobables de 0 y 1, por lo cual no puede comprimirse más.

Si en cambio $p \rightarrow 0$, la fuente tiene secuencias largas de 0, y en ese caso puede ser más eficiente codificar el largo de la cadena de 0s que indicar cada bit por separado. En el caso extremo, solo alcanza 1 bit para toda la cadena ("son todos ceros"). Análogamente, si $p \rightarrow 1$.

1.3.2. Codificación de canal

El objetivo de la codificación de fuente es reducir la cantidad de datos a transmitir a su mínima expresión, eliminando redundancia sin pérdida de información (o en el caso con pérdida, sin distorsión relevante). La *codificación de canal* va en sentido opuesto: agrega redundancia para anticiparse a posibles errores del canal de transmisión y poder detectarlos y corregirlos. A continuación damos algunos ejemplos.

Ejemplo 1.3 (Bit de paridad). Supongamos una fuente binaria $\{a_k\}$ y formemos bloques de largo N . Al final de cada bloque se agrega un bit $a_N = a_0 \oplus \dots \oplus a_{N-1}$ donde $\oplus$ es la operación XOR. Luego se transmite la palabra $b = (a_0, \dots, a_N)$. El resultado final es que $a_N = 1$ si hay una cantidad impar de 1s en el bloque, o dicho de otro modo, cada bloque b tiene una cantidad par de bits. Esto permite detectar (pero no corregir) errores de 1 bit, con un overhead de $1/N$ bits por símbolo.

Obviamente, si el largo de bloque N es grande, el overhead es mínimo, pero la probabilidad de que haya errores de más de un bit es mayor, por lo cual la elección de N es un compromiso entre ambos efectos.

Ejemplo 1.4 (Código de repetición). Supongamos una fuente binaria $\{a_k\}$ y realicemos el siguiente procedimiento: para cada símbolo a_k formamos un bloque $b = (a_k, a_k, a_k)$. Esto es un código de repetición de largo 3. Observemos que si hay un error de 1 bit, podemos decidir por mayoría si el bloque corresponde a un 0 o a un 1. Por ejemplo $(110) \mapsto 1$ y $(010) \mapsto 0$. Esto permite detectar y corregir errores de 1 bit. Es sin embargo muy ineficiente requiriendo 3 bits por símbolo.

Ejemplo 1.5 (Matriz de paridad). Consideremos ahora el siguiente procedimiento para una fuente binaria $\{a_k\}$. Elegimos un número M y formamos bloques de tamaño $N = M^2$ dispuestos en forma de matriz de $M \times M$. Luego se agrega a dicha matriz un bit de paridad para cada fila y cada columna, así como un bit de paridad total. Por ejemplo, si $M = 4$, $N = 16$:

$$(1001000111111011) \mapsto \begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \end{pmatrix} \mapsto \begin{pmatrix} 1 & 0 & 0 & 1 & \vdots & 0 \\ 0 & 0 & 0 & 1 & \vdots & 1 \\ 1 & 1 & 1 & 1 & \vdots & 0 \\ 1 & 0 & 1 & 1 & \vdots & 1 \\ \dots & \dots & \dots & \dots & \dots & \dots \\ 1 & 1 & 0 & 0 & \vdots & 0 \end{pmatrix}$$

Luego se envían los N bits y los $M + 1$ bits de paridad agregados.

Si hay un error de un bit, el mismo se detecta y se corrige porque los errores de paridad indican la fila y columna del error, por ejemplo:

$$\begin{pmatrix} 1 & 0 & 0 & 1 & \vdots & 0 \\ 0 & 0 & 0 & 1 & \vdots & 1 \\ 1 & 1 & 1 & 1 & \vdots & 0 \\ 1 & 0 & 1 & 1 & \vdots & 1 \\ \dots & \dots & \dots & \dots & \dots & \dots \\ 1 & 1 & 0 & 0 & \vdots & 0 \end{pmatrix} \rightsquigarrow \begin{pmatrix} 1 & 0 & 0 & 1 & \vdots & 0 \\ 0 & 0 & \color{red}{1} & 1 & \vdots & \color{red}{1*} \\ 1 & 1 & 1 & 1 & \vdots & 0 \\ 1 & 0 & 1 & 1 & \vdots & 1 \\ \dots & \dots & \dots & \dots & \dots & \dots \\ 1 & 1 & \color{red}{0*} & 0 & \vdots & 0 \end{pmatrix}$$

El código detecta entonces donde ocurrió el error porque la paridad de la fila y la columna no es correcto. Notar que el error también puede ocurrir en la fila o en la columna de paridad. Esto también se detecta y se corrige gracias al bit extra de paridad total.

El resultado es que se envían N bits con una redundancia de $2M + 1 = 2\sqrt{N} + 1$ por lo que en un bloque de $N = 1024$ bits solo se requieren 33 bits extra. El overhead crece más lento que el tamaño de bloque. De todos modos sigue siendo ineficiente y solo permite corregir errores de 1 bit, por lo que si el tamaño de bloque es grande, tenemos el mismo problema que en Ejemplo 1.3.

El punto común de todos los ejemplos anteriores es el siguiente: dado un alfabeto de símbolos (por ejemplo binario), construimos bloques de largo N . Esto permite generar 2^N símbolos posibles. Luego agregamos bits de control, generando palabras de largo $M > N$, y aplicando un código que lleva secuencias de largo N a un espacio mayor de largo M , donde los códigos “válidos” se encuentran *más separados entre sí* (solo hay 2^N palabras válidas de las 2^M posibles). Cuando ocurre algún error, encontramos una palabra no válida y detectamos el error. Si el código es adecuado, podemos además mapear esa palabra no válida a un símbolo válido corrigiendo entonces el error.

A mayor M , más redundancia se agrega, más espacio se genera, y podemos detectar y corregir errores más grandes. Pero por otra parte, el overhead es mayor. Se le denomina *tasa de codificación* (code rate) al cociente N/M .

Un ejemplo de código de este tipo son los [Códigos de Hamming](#), en los cuales se elige un tamaño de bloque $M = 2^r - 1$ con $r \geq 1$ y se envía un mensaje de largo $N = 2^r - r - 1$ con r bits de redundancia, utilizando una transformación lineal invertible. En ese caso, la tasa de codificación queda:

$$R = 1 - \frac{r}{2^r - 1},$$

es decir, la redundancia es del orden de $\log_2(N) \ll \sqrt{N}$. Sin embargo, si bien es más eficiente que la matriz de paridad, este código solo permite detectar errores de 1 o 2 bits, y corregir errores de un único bit. El tipo más común de código de bloques es el de [Reed–Solomon, 1960](#), una familia de códigos que permite detectar errores de hasta $k = M - N$ bits y corregir hasta $\lfloor k/2 \rfloor$ bits, siendo k un parámetro, la cantidad de redundancia deseada. Es además capaz de recuperar errores de “borrado”, donde no es posible decidir si el bit recibido es 0 o 1.

Más recientemente, la aparición de los [códigos convolucionales](#), [Turbo–codes](#) y códigos [Low–Parity–Data–Check](#), se logran tasas de codificación cercanas a 1 con muy baja probabilidad de error.

Capítulo 2

Procesos aleatorios

En esta sección

Apéndice A

Apéndice